

人工智能 第10回

多くの層から構成される階層型のニューラルネットワークの総称

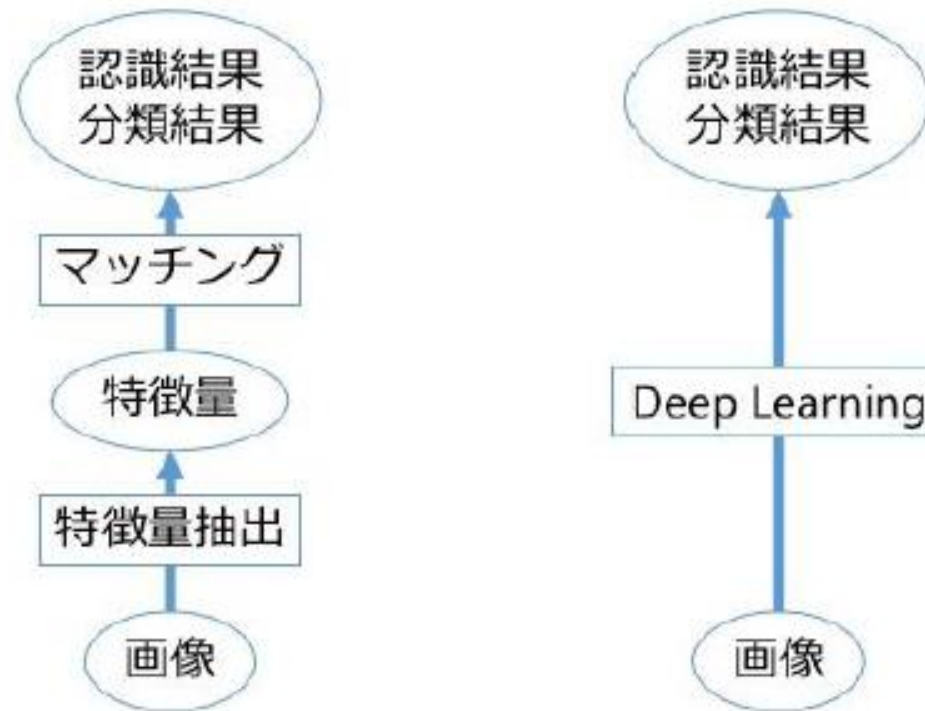
2006年 Hinton and Salakhutdinov, Science, 2006あたりから

2010年 7～8層程度

2014年 20層以上のものも...

音声認識・画像認識・自然言語処理など様々な分野で圧倒的な性能を実現

特徴量の抽出を事前にしなくてもよい!?



Google, Facebook, Yahoo など注目して盛んに研究

80年代後半～90年代半ば

誤差逆伝播法

3層のネットワークではうまくいったが、それ以上の多層のネットワークになると期待したような結果が得られなかった.

- ・ 汎化能力(未学習のデータに対する能力)が上がらない
- ・ 過学習(学習データに対してのみ必要以上に学習してしまう状態)が発生してしまう

誤差(E)を重み(w)で偏微分した勾配に基づいて重みを更新するが、出力層から離れるにつれて

- 勾配が急激に小さくなったり

- 大きくなって発散してしまったりする

ことがあり、層の数が多くなると重みが適切に更新できないという問題があった。

畳み込みニューラルネットワークなどの階層型ニューラルネットワークなどの
学習方法の最も基本的なもの

【勾配降下法】 $\approx$ 最急降下法（誤差逆伝播法）

すべての訓練データに対する誤差を求め、それを用いて重みを更新する

《問題点》

誤差関数の勾配を降下する方向に重みを更新していくため、重みの初期値によっては誤差関数の最小値に到達できない(ローカルミニマムにはまってしまうと抜け出せない)

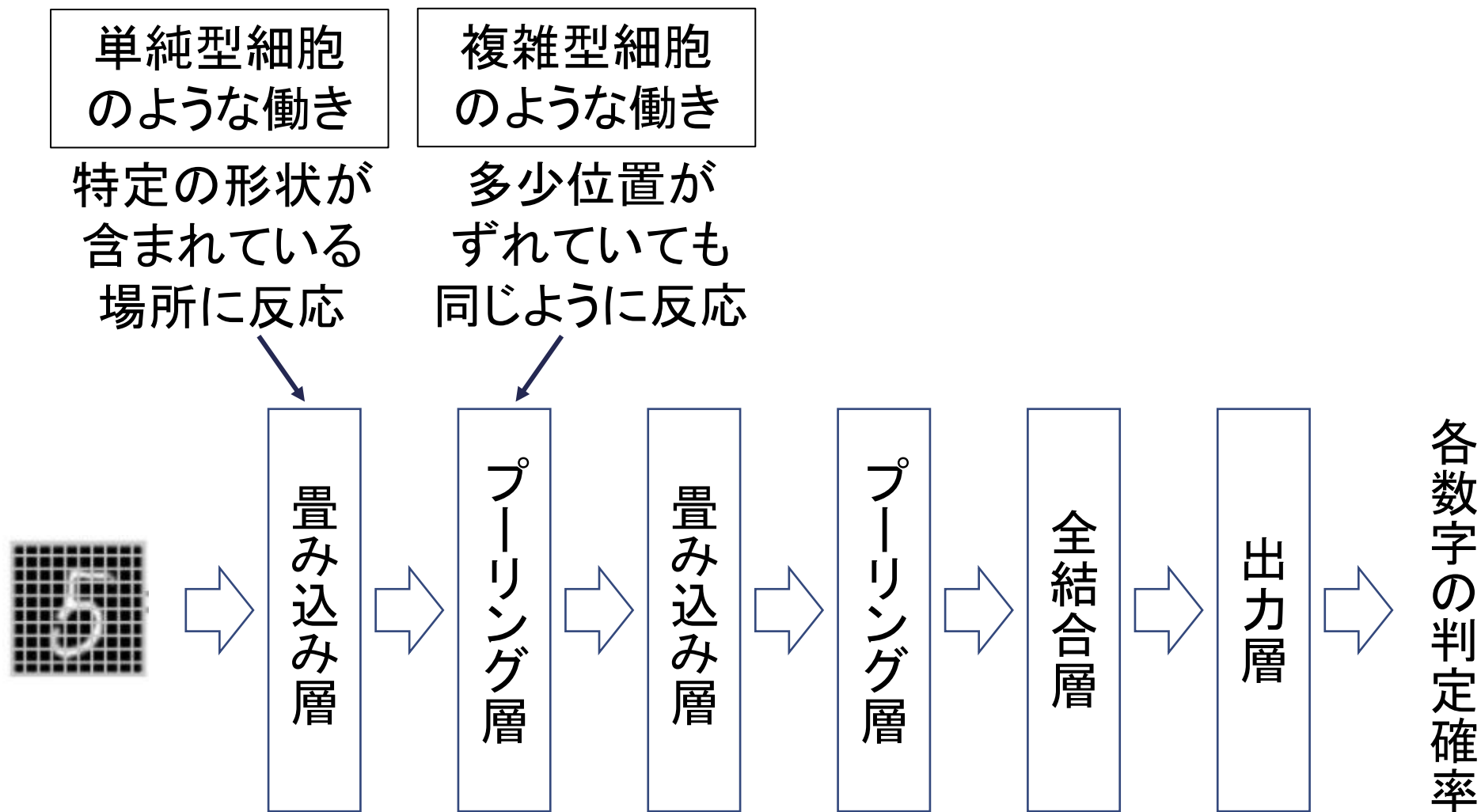
➔ 確率的勾配降下法

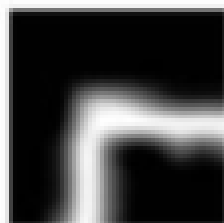
Convolutional Neural Network : CNN

- ・ディープラーニング(深層学習)の代表的な手法
- ・画像認識などに有効

生物の視覚情報処理における知見との類似性も...

- ・単純型細胞 : 特定の向きの線に反応するニューロン
- ・複雑型細胞: 受容野(ニューロンが結合している範囲)のどこの位置にあっても反応するニューロン
 - 少しくらい位置がずれていても同じように扱える

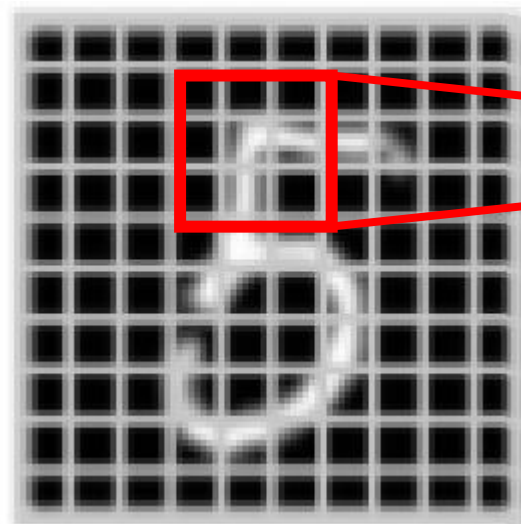




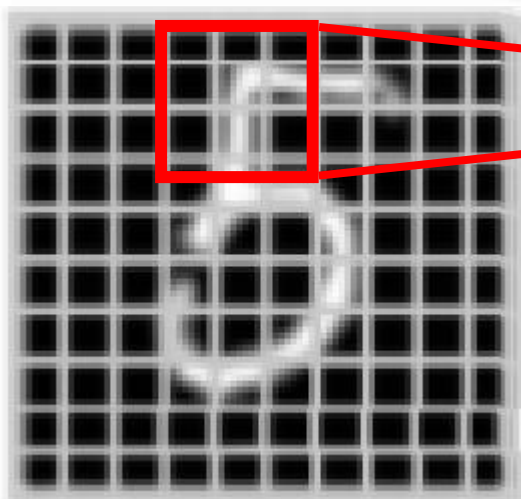
特定の形状が
含まれている
場所が反応

※ 実際には色々な
形状に対して
同じ処理を行う

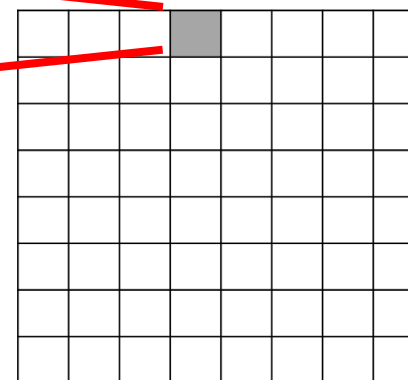
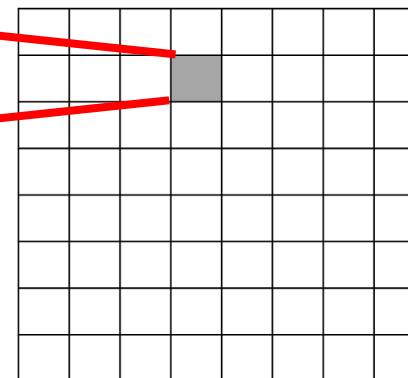
入力
1



入力
2



畳み込み層



関数を平行移動しながら関数に重ね足し合わせる二項演算

画像における畳み込み演算は、データに含まれるノイズを緩和したり、情報を抽出したりするために用いられる

入力画像に対して $H \times H$ のサイズのフィルタを用いて畳み込み演算を行ったときに得られる画像の画素 (i, j) の画素値 u_{ij}

$$u_{ij} = \sum_{p=0}^{H-1} \sum_{q=0}^{H-1} x_{i+p, j+q} h_{pq}$$

x_{ij} : 畳み込みを行う画像の画素 (i, j) の画素値

h_p : フィルタの画素 (p, q) の画素値

$$u_{ij} = \sum_{p=0}^{H-1} \sum_{q=0}^{H-1} x_{i+p,j+q} h_{pq}$$

$H = 3, i = 2, j = 3$ とすると

$$\begin{aligned} u_{23} &= x_{2+0,3+0}h_{00} + x_{2+0,3+1}h_{01} + x_{2+0,3+2}h_{02} \\ &\quad + x_{2+1,3+0}h_{10} + x_{2+1,3+1}h_{11} + x_{2+1,3+2}h_{12} \\ &\quad + x_{2+2,3+0}h_{20} + x_{2+2,3+1}h_{21} + x_{2+2,3+2}h_{22} \\ &= x_{23}h_{00} + x_{24}h_{01} + x_{25}h_{02} \\ &\quad + x_{33}h_{10} + x_{34}h_{11} + x_{35}h_{12} \\ &\quad + x_{43}h_{20} + x_{44}h_{21} + x_{45}h_{22} \end{aligned}$$

$$u_{ij} = \sum_{p=0}^{H-1} \sum_{q=0}^{H-1} x_{i+p, j+q} h_{pq}$$

$$H = 3, \quad i = 2, \quad j = 3$$

$$u_{23} = x_{23}h_{00} + x_{24}h_{01} + x_{25}h_{02} + x_{33}h_{10} + x_{34}h_{11} + x_{35}h_{12} \\ + x_{43}h_{20} + x_{44}h_{21} + x_{45}h_{22}$$

入力画像 x_{ij}

		0	1	2	3	4	5	6
0								
1								
2				31	0	1		
3				2	30	0		
4				1	2	33		
5								
6								

フィルタ h_{pq}

					q	
				θ	1	2
		θ		1	θ	θ
$\otimes$	p	1		θ	1	θ
		2		θ	θ	1



p

=

出力画像 u_{ij}

		0	1	2	3	4
0						
1						
2				94		
3						
4						

$N_W \times N_H$ のサイズの画像に対し、 $H \times H$ のサイズのフィルタを適用した場合に
得られる画像のサイズ

$$(N_W - H + 1) \times (N_H - H + 1)$$

入力画像 x_{ij}

		j						
		0	1	2	3	4	5	6
i	0							
	1							
	2							
	3							
	4							
	5							
	6							

フィルタ h_{pq}

		q		
		0	1	2
p	0			
	1			
	2			



=

出力画像 u_{ij}

		j				
		0	1	2	3	4
i	0					
	1					
	2					
	3					
	4					

入力画像 x_{ij}

38	2	1	3	2
0	31	0	0	1
0	1	27	3	3
1	2	1	25	2
2	1	20	2	32

フィルタ h_{pq}

1	0	0
0	1	0
0	0	1

 $=$ 出力画像 u_{ij}

96	5	4
2	83	5
22	4	84

フィルタに含まれる特徴が含まれる部分が強く反応

畳み込み演算の前後で同じサイズの画像を得たい場合には、入力画像の周囲に幅 $\lfloor H/2 \rfloor$ のふちを追加(パディング)した画像を用いる。

ゼロパディング : 外側に追加するふちの部分をすべて0にするパディング

入力画像 x_{ij}

0	0	0	0	0	0	0
0	38	2	1	3	2	0
0	0	31	0	0	1	0
0	0	1	27	3	3	0
0	1	2	1	25	2	0
0	2	1	20	2	32	0
0	0	0	0	0	0	0

フィルタ h_{pq}

1	0	0
0	1	0
0	0	1



=

出力画像 u_{ij}

69	2	1	4	2
1	96	5	4	4
2	2	83	5	3
2	22	4	84	5
2	2	22	3	57

畳み込み演算においてフィルタを適用する場合には、フィルタを1画素ずつずらしながら適用しているが、数画素ずつずらしながら適用することもある。このような場合のフィルタの適用間隔をストライドという。

入力画像 x_{ij}

	j						
	0	1	2	3	4	5	6
0							
1							
2							
3							
4							
5							
6							

フィルタ h_{pq}

	q		
	0	1	2
0			
1			
2			

$\otimes$ p

出力画像 u_{ij}

	j		
	0	1	2
0			
1			
2			

ストライド2

入力画像 x_{ij}

38	2	1	3	2
0	31	0	0	1
0	1	27	3	3
1	2	1	25	2
2	1	20	2	32

フィルタ h_{pq}

1	0	0
0	1	0
0	0	1



=

出力画像 u_{ij}

96	4
22	84

ストライド2

- ・ 前の層の一部のニューロン(受容野内のニューロン)とのみ結合
- ・ 受容野内のニューロンと結合する重みがフィルタを表しており、受容野内のニューロンの出力がフィルタのパターンと類似しているとき、受容野と結合している畳み込み層のニューロンが活性化

前の層の出力(最初の層の場合には入力)のどのあたりにどのような(形の)特徴が含まれているかを抽出する層

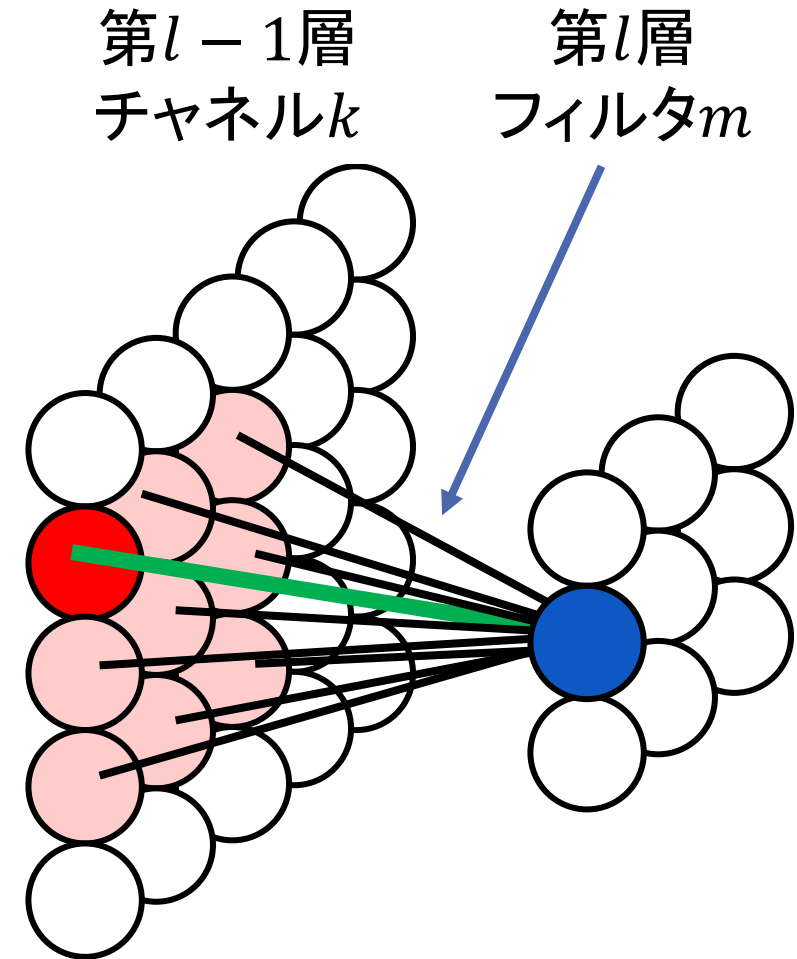
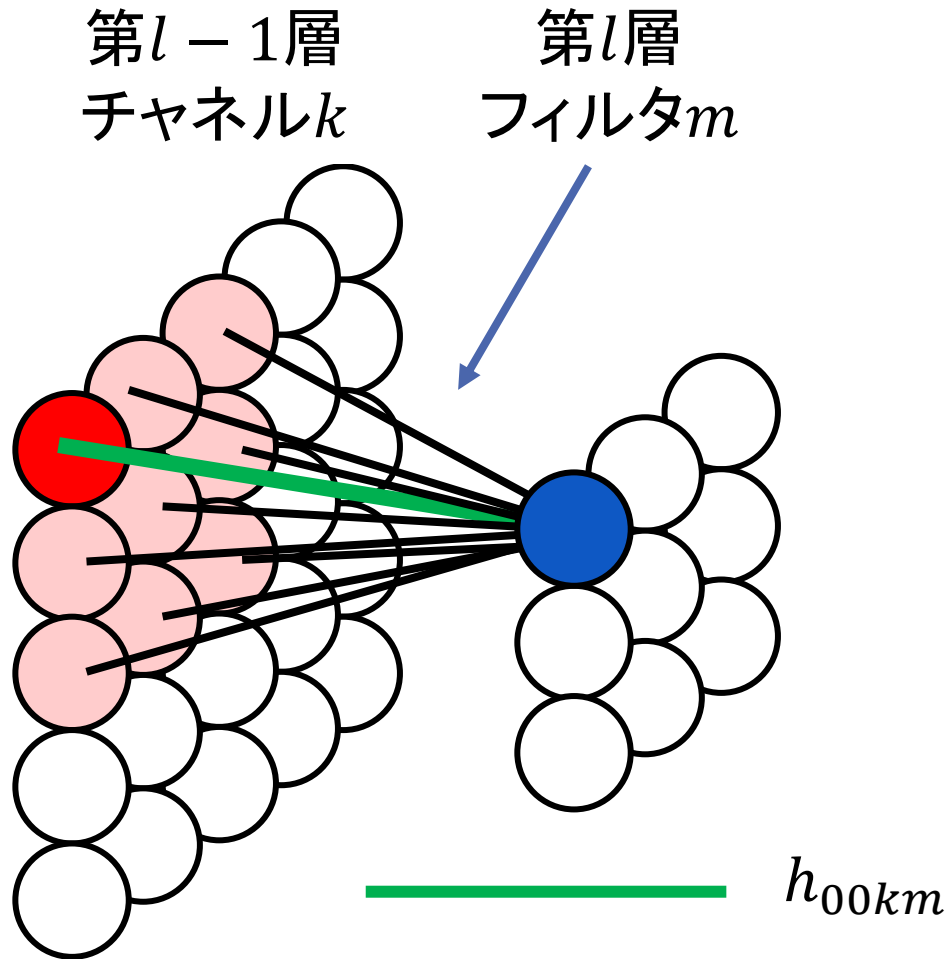
※ 抽出する特徴(フィルタ)は学習によって自動的に決定

K 個のチャネルを持つ第 $l-1$ 層の出力を入力として受け取る第 l 層のフィルタ m に対応するニューロン (i, j) の内部状態 $u_{ijm}^{(l)}$

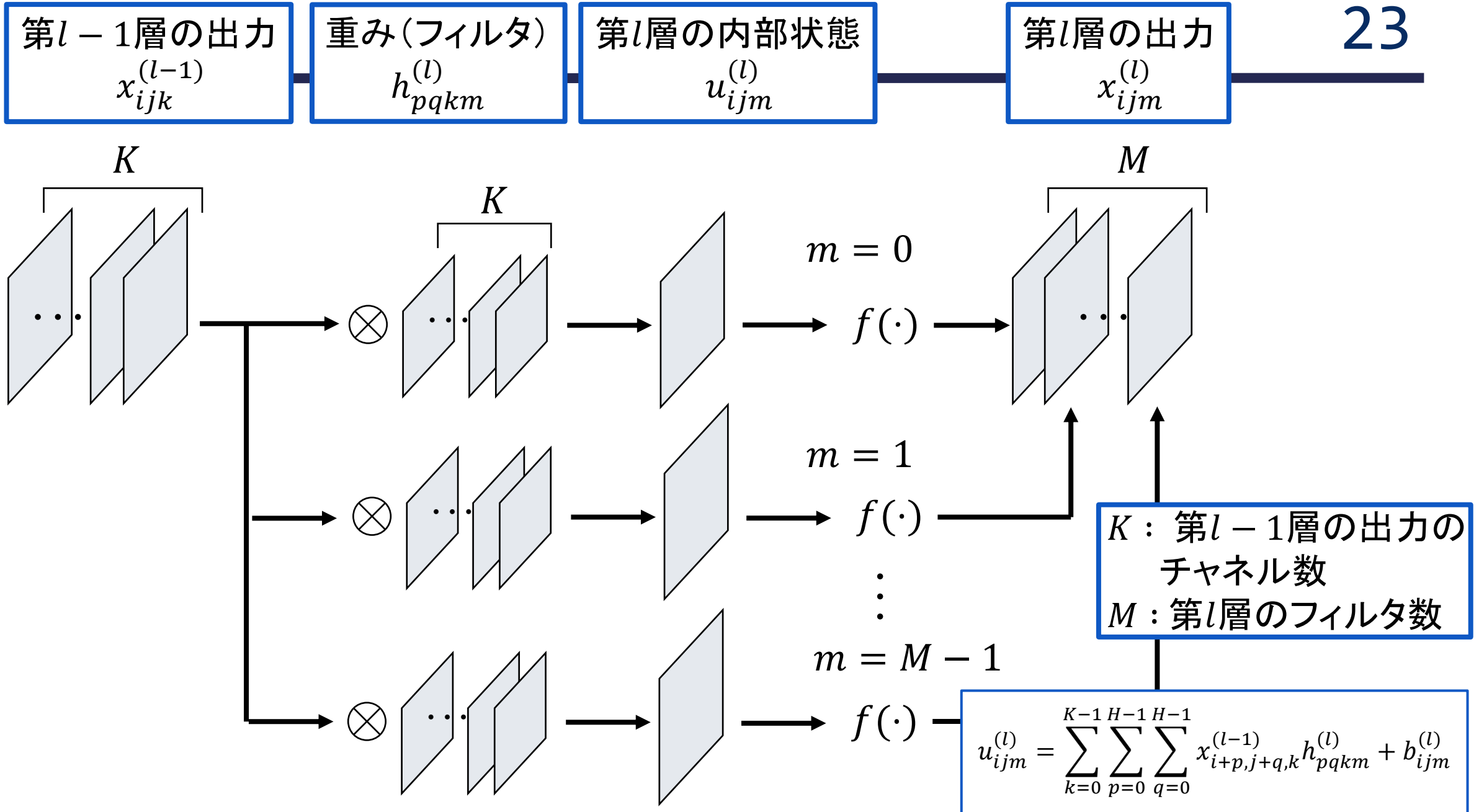
$$u_{ijm}^{(l)} = \sum_{k=0}^{K-1} \sum_{p=0}^{H-1} \sum_{q=0}^{H-1} x_{i+p, j+q, k}^{(l-1)} h_{pqkm}^{(l)} + b_{ijm}^{(l)}$$

畳み込み演算
(チャネルの分だけ加算)

- H : 受容野の1辺のサイズ (フィルタの1辺のサイズ)
- $x_{ijk}^{(l-1)}$: 第 $l-1$ 層のチャネル k に対応するニューロン (i, j) の出力
- $h_{pqkm}^{(l)}$: 第 $l-1$ 層のチャネル k に対応する受容野の (p, q) の位置にあるニューロンから第 l 層のフィルタ m に対応するニューロンへの重み
- $b_{ijm}^{(l)}$: 第 l 層のフィルタ m に対応するニューロン ij のバイアス (しきい値と同じ役割(符号は逆))



重み共有：
フィルタの同じ部分にあたる重みは同じ値



信号のチャンネル数(K)は何によって決まる?

入力がRGB画像であればRGBの3チャンネル

第2層以降であれば、前の層の出力のチャンネル数

層の出力のチャンネル数は...

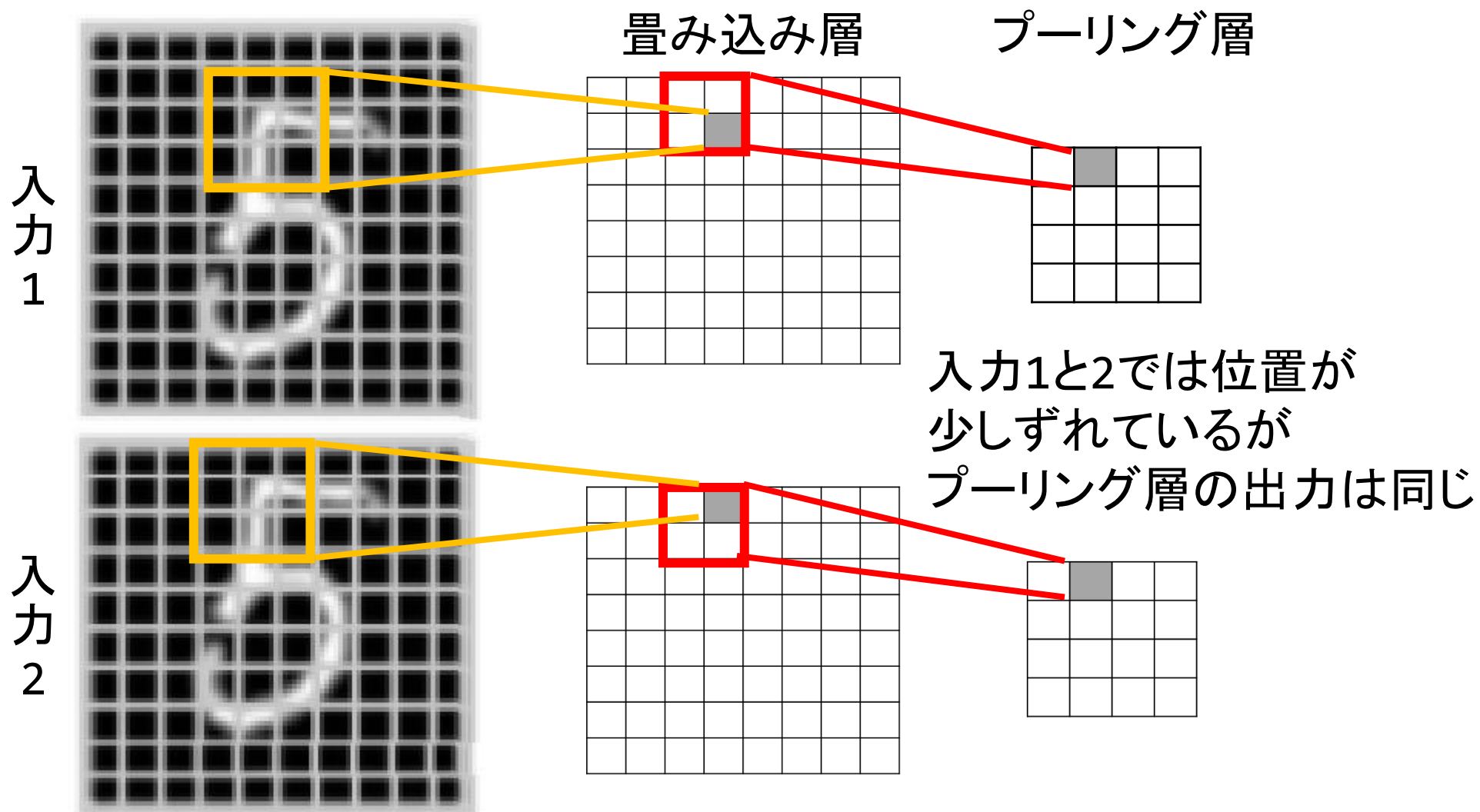
その層のフィルタの数(M)と等しい

第 l 層のフィルタ m に対応するニューロン (i, j) の出力 $x_{ijm}^{(l)}$

$$x_{ijm}^{(l)} = f(u_{ijm}^{(l)})$$

$$f(u) = \max(0, u)$$

正規化線形関数 (Rectified Linear Unit : ReLU)



直前の畳み込み層の出力に対してプーリングを行う

- ・ 畳み込み層の出力はその前の層に含まれる特徴的なパターンを抽出したものになっており、どの位置にどのようなパターンが含まれているかという大まかな特徴を表現
- ・ プーリング層のニューロンは受容野内のニューロンの出力に応じて出力が決定するので、直前の畳み込み層における出力が多少異なっても、受容野内の他のニューロンが同じように発火していればプーリング層の出力は類似
→ 畳み込み層で出力された情報の位置に関する感度を低下させる働き

【最大プーリング (Max Pooling)】 ※ 画像認識などでは最大プーリングを
使うことが多い

第 l 層のチャネル k に対応するニューロン (i, j) の内部状態 $u_{ijk}^{(l)}$

$$u_{ijk}^{(l)} = \max_{(p,q) \in P_{ijk}^{(l)}} x_{pqk}^{(l-1)}$$

第 l 層のチャネル k に対応するニューロン (i, j) の出力 $x_{ijk}^{(l)}$

$$x_{ijk}^{(l)} = u_{ijk}^{(l)}$$

$P_{ijk}^{(l)}$: 第 l 層のチャネル k に対応するニューロン (i, j) の
受容野のニューロンの集合

$x_{pqk}^{(l-1)}$: 第 $l-1$ 層のチャネル k に対応するニューロン (p, q)
の出力

【平均プーリング (Average Pooling)】

第 l 層のチャネル k に対応するニューロン (i, j) の内部状態 $u_{ijk}^{(l)}$

$$u_{ijk}^{(l)} = \frac{1}{|P_{ijk}^{(l)}|} \sum_{(p,q) \in P_{ijk}^{(l)}} x_{pqk}^{(l-1)}$$

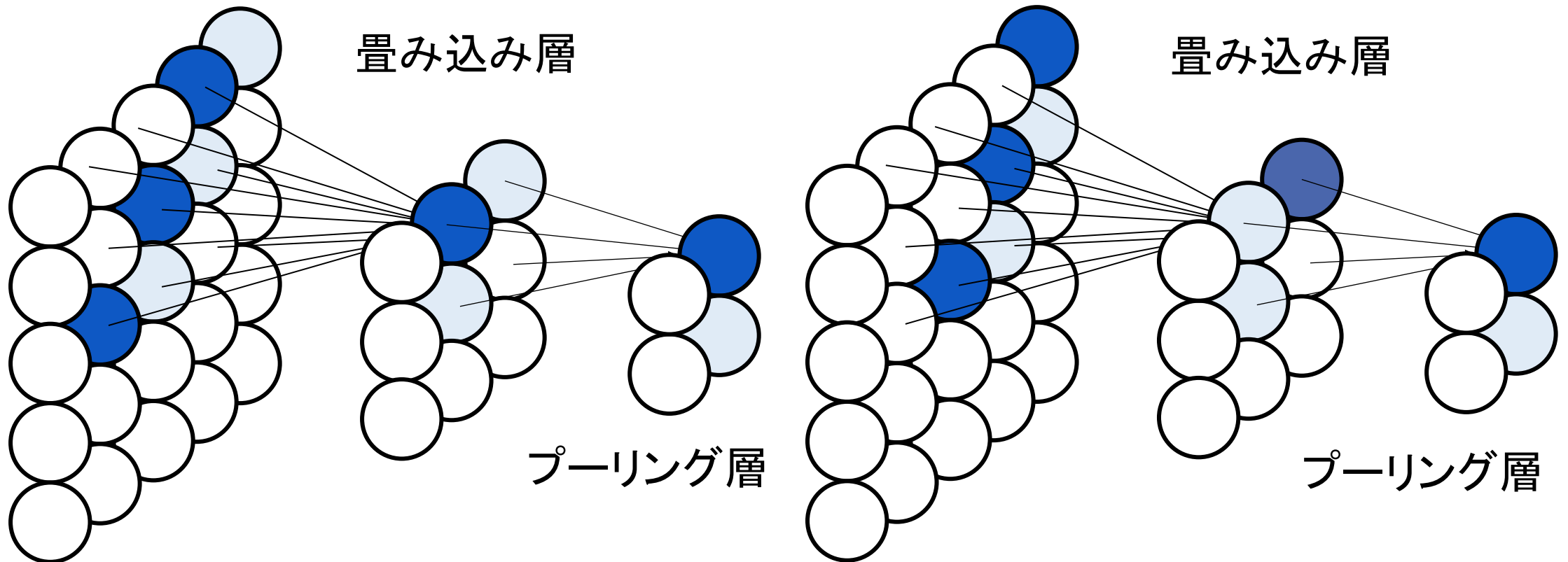
第 l 層のチャネル k に対応するニューロン (i, j) の出力 $x_{ijk}^{(l)}$

$$x_{ijk}^{(l)} = u_{ijk}^{(l)}$$

$P_{ijk}^{(l)}$: 第 l 層のチャネル k に対応するニューロン (i, j) の
受容野のニューロンの集合

$x_{pqk}^{(l-1)}$: 第 $l-1$ 層のチャネル k に対応するニューロン (p, q)
の出力

最大プーリングの場合



畳み込み層で発火しているニューロンの位置が少し違っても
プーリング層の出力は同じになる

- ・ 直前の層のすべてのニューロンから信号を受け取る層

畳み込み層やプーリング層では受容野内のニューロンとしか結合していないので局所的な特徴しかみていない。最終的な認識などを行うためには、全体の特徴を総合的に判断する必要があるので全結合層が必要

第 l 層のニューロン i の内部状態 $u_i^{(l)}$

$$u_i^{(l)} = \sum_{j=1}^{N^{(l-1)}} w_{ij}^{(l)} x_j^{(l-1)} + b_i^{(l)}$$

第 l 層のニューロン i の出力 $x_i^{(l)}$

$$x_i^{(l)} = f(u_i^{(l)})$$

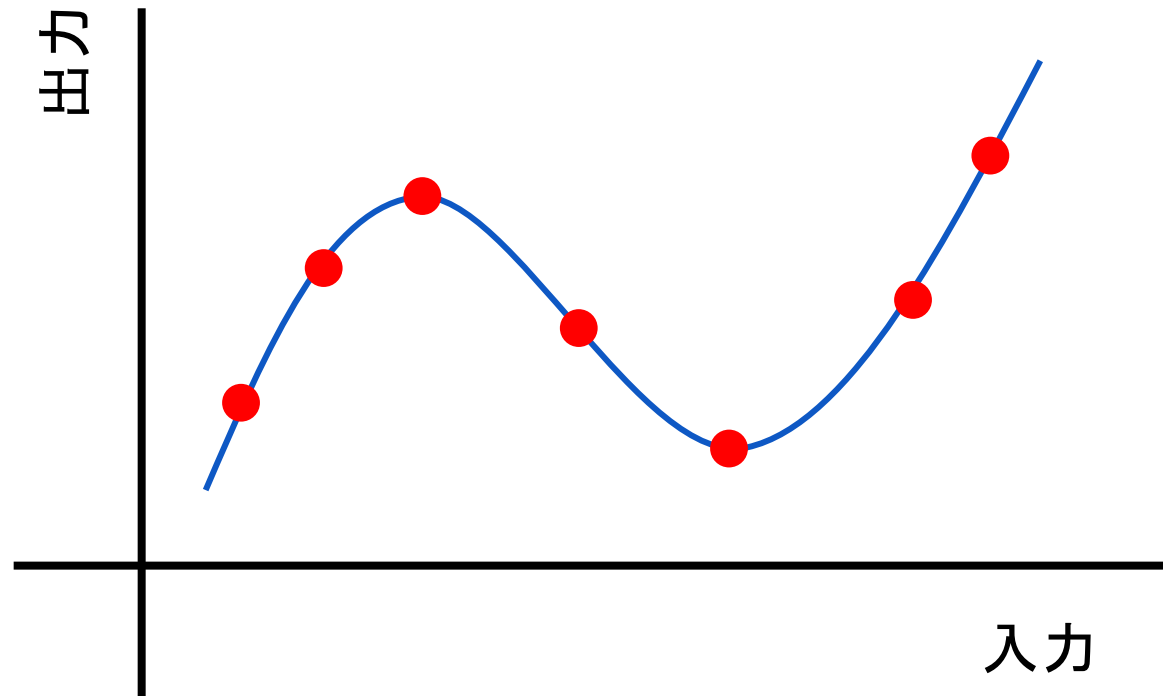
※ 出力関数としてはReLUが用いられることが多い

-
- ・ 出力層も全結合層の一種
 - ・ 扱いたい問題に応じて出力層のニューロンの出力関数や誤差関数が決まる

回帰(regression)問題の場合

33

(主に)出力に連続値をとる関数を対象に訓練データをよく再現するような関数を定める問題



【出力 (教師信号)】

任意の実数値

【出力層の出力関数】 出力層の出力の取り得る値により選択
-1~1 なら 双曲線正接関数 (tanh)

$$f(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}}$$

0~1なら シグモイド関数

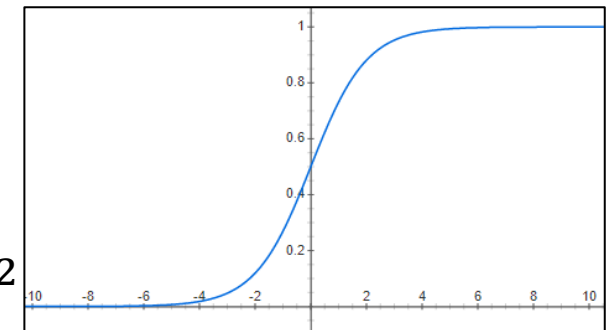
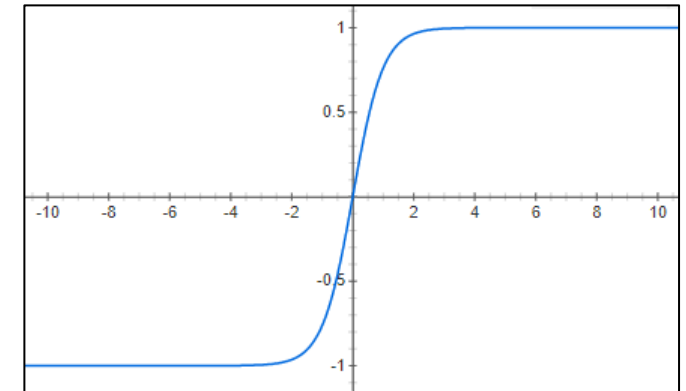
$$f(u) = \frac{1}{1 + e^{-u}}$$

任意の実数なら 恒等写像

$$f(u) = u$$

【誤差関数】 2乗誤差

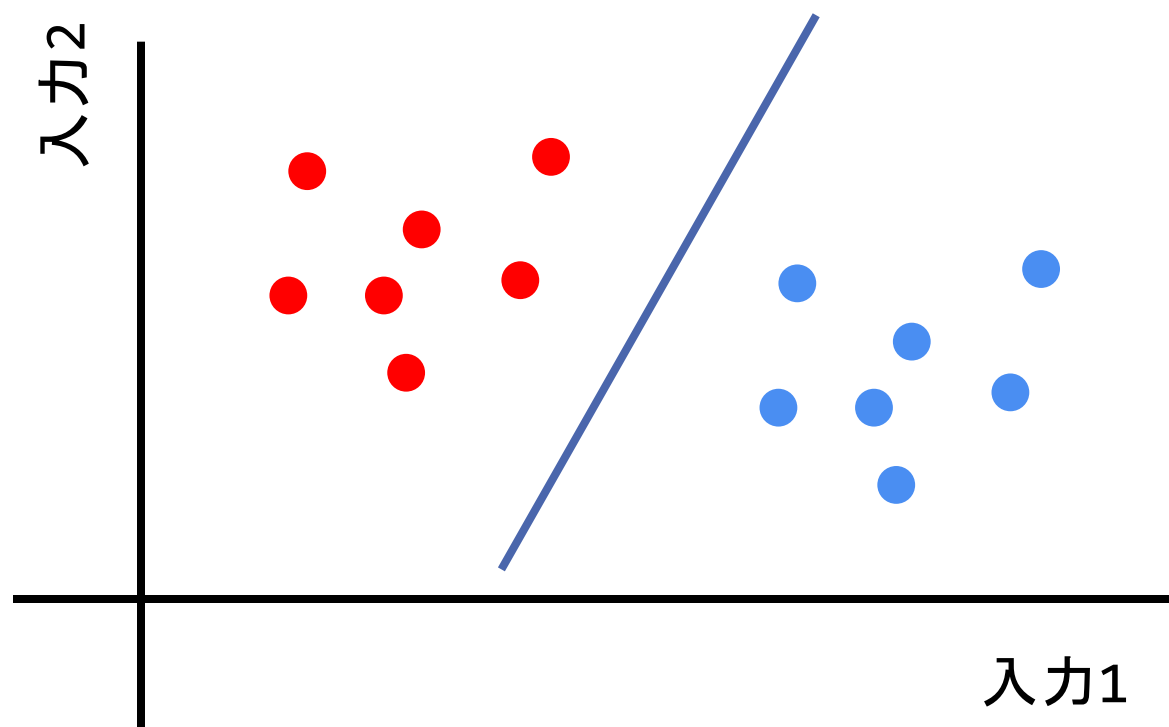
$$E(\mathbf{w}) = \frac{1}{2} \sum_{p=1}^P \sum_{k=1}^K \left(d_k^{(p)} - x_k^{O(p)} \right)^2$$



2値分類 (2クラス分類)の場合

35

入力を内容に応じて2種類に区別(分類)する問題



【出力(教師信号)】

$$d = \{0, 1\}$$

2つのクラスのどちらに属するかを 0 or 1 で表現

【出力関数】

シグモイド関数

$$f(u) = \frac{1}{1 + e^{-u}}$$

【誤差関数】

$$E(\mathbf{w}) = - \sum_{p=1}^P [d^{(p)} \log x^{O(p)} + (1 - d^{(p)}) \log (1 - x^{O(p)})]$$

$x^{(p)}$ が与えられたとき $d = 1$ となる事後確率 $p(d = 1 | x^{(p)})$ をモデル化する。

モデル化 = ニューラルネットワークに $x^{(p)}$ が入力されたときに

$p(d = 1 | x^{(p)})$ を出力するように学習する(重み $\mathbf{w}$ を調整する)

出力層のニューロンの出力が 0.5以上なら1、0.5未満なら0とする

$x^O(x^{(p)}; \mathbf{w})$: 重みが $\mathbf{w}$ のネットワークに $x^{(p)}$ を与えたときの出力

多クラス分類の場合

38

入力を内容に応じて複数のクラスに区別(分類)する問題
3クラス以上の場合を多クラス分類という

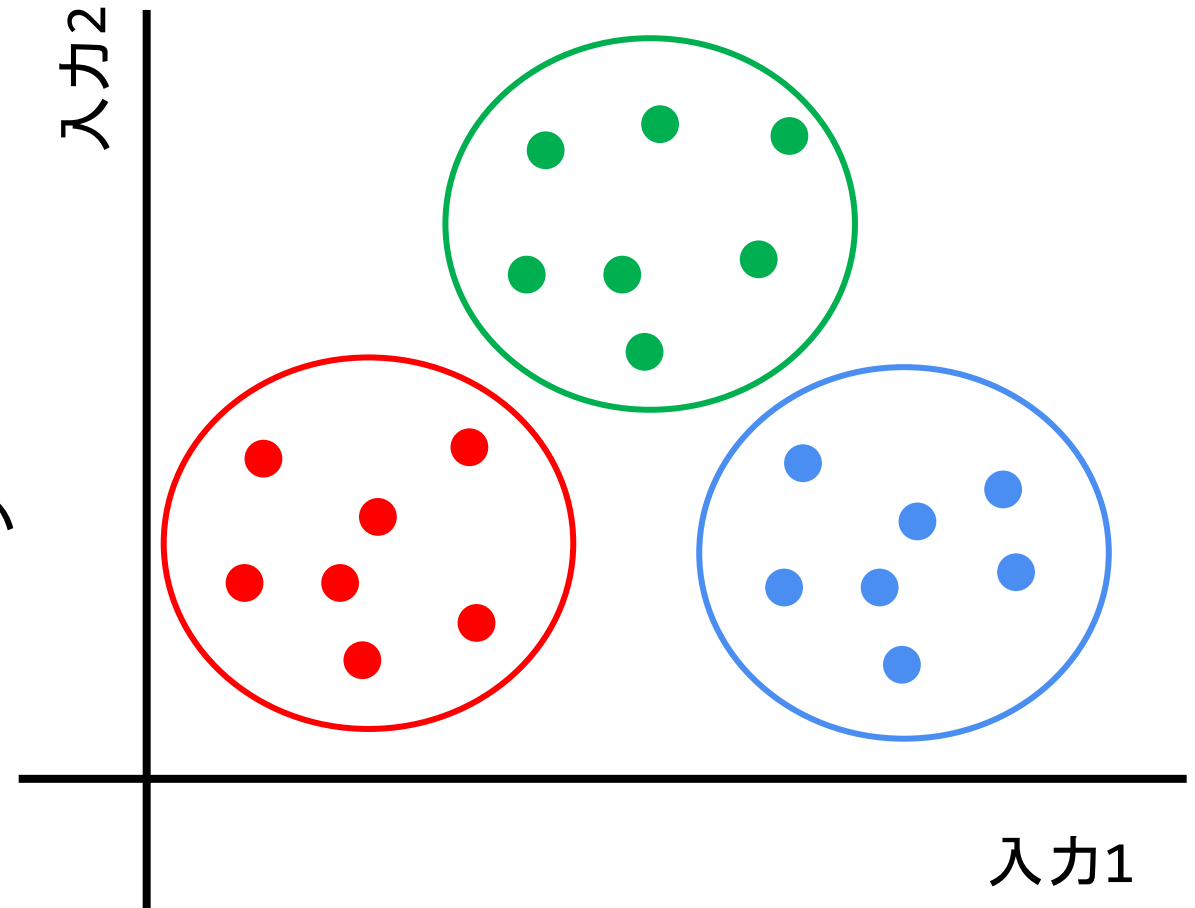
(例) 手書き数字の認識

入力 :

数字が1つ書かれた画像(の画素値)

出力 :

0~9の10のクラスのいずれに属するか



【出力(教師信号)】

- ・ 出力層のニューロン数は分類したいクラスの数
- ・ 各ニューロンは各クラスに対応しており、ニューロン k は入力 $x^{(p)}$ がクラス k に属する確率を出力
- ・ 教師信号は対応するクラスのみ値が1、それ以外は0

(例)

クラス数が5で入力 $x^{(p)}$ がクラス3に属するとき

$$\mathbf{d}^{(p)} = [0 \ 0 \ 1 \ 0 \ 0]^T$$

クラス 1 2 3 4 5

【出力関数】

ソフトマックス関数

$$x_k^O = f(u_k^O) = \frac{\exp(u_k^O)}{\sum_{j=1}^K \exp(u_j^O)}$$

【誤差関数】 クロスエントロピー誤差

$$E(\mathbf{w}) = - \sum_{p=1}^P \sum_{k=1}^K d_k^{(p)} \log x_k^{O(p)}$$

$x^{(p)}$ が与えられたとき d となる事後確率 $p(d^{(p)} | x^{(p)})$ をモデル化する。

モデル化 = ニューラルネットワークに $x^{(p)}$ が入力されたときに

$p(d^{(p)} | x^{(p)})$ を出力するように学習する(重み w を調整する)

$x^O(x^{(p)}; \mathbf{w})$: 重みが w のネットワークに $x^{(p)}$ を与えたときの出力

確率的勾配降下法 (Stochastic Gradient Descent : SGD)

42

【確率的勾配降下法】

1つ1つの訓練データに対する誤差を求め、それを用いて重みを更新する

《メリット》

誤差関数が訓練データごとに異なるため、ローカルミニマムにはまりにくい

《問題点》

最終的に最小化したいのはすべての訓練データに対する誤差関数であるが、
毎回、異なる誤差関数に対して最小化しようとする更新が行われるため、
学習の収束に時間がかかる可能性がある

【ミニバッチ確率的勾配降下法】（これを確率的勾配降下法ということも多い）

訓練データをいくつかのミニバッチに分けてそれに対する誤差を求め、
それを用いて重みを更新する

《メリット》

ミニバッチごとに誤差関数が異なるので、ローカルミニマムにはまりにくい

訓練データごとに異なる誤差関数を最小化する場合に比べると、ミニバッチ
ごとの誤差関数は訓練データ全体に対する誤差関数と似たような形になって
いる可能性が高く、学習がはやく進む可能性がある

※ 以下の説明ではミニバッチ確率的勾配降下法を確率的勾配降下法と呼ぶ

重みの更新式

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \frac{\partial E_s^{(t)}}{\partial \mathbf{w}^{(t)}}$$

ミニバッチ s に関する誤差

第 l 層の重み $w_{ji}^{(l,t)}$ についての誤差関数 $E_s^{(t)}$ の微分

$$\begin{aligned} \frac{\partial E_s^{(t)}}{\partial w_{ji}^{(l,t)}} &= \sum_{p \in C^s} \frac{\partial E_s^{(t,p)}}{\partial u_j^{(l,t,p)}} \frac{\partial u_j^{(l,t,p)}}{\partial w_{ji}^{(l,t)}} \\ &= \sum_{p \in C^s} \sum_{k=1}^{N^{(l+1)}} \frac{\partial E_s^{(t,p)}}{\partial u_k^{(l+1,t,p)}} \frac{\partial u_k^{(l+1,t,p)}}{\partial x_j^{(l,t,p)}} \frac{\partial x_j^{(l,t,p)}}{\partial u_j^{(l,t,p)}} \frac{\partial u_j^{(l,t,p)}}{\partial w_{ji}^{(l,t)}} \end{aligned}$$

第 l 層のニューロン j の内部状態についての誤差関数 $E_s^{(t)}$ の微分 $\delta_j^{(l,t)}$

$$\delta_j^{(l,t)} = \sum_{p \in C^s} \frac{\partial E_s^{(t,p)}}{\partial u_j^{(l,t,p)}} \quad \text{とおくと} \quad \delta_j^{(l+1,t)} = \sum_{p \in C^s} \frac{\partial E_s^{(t,p)}}{\partial u_j^{(l+1,t,p)}} \quad \text{と}$$

$$\frac{\partial E_s^{(t)}}{\partial w_{ji}^{(l,t)}} = \sum_{p \in C^s} \sum_{k=1}^{N^{(l+1)}} \frac{\partial E_s^{(t,p)}}{\partial u_k^{(l+1,t,p)}} \frac{\partial u_k^{(l+1,t,p)}}{\partial x_j^{(l,t,p)}} \frac{\partial x_j^{(l,t,p)}}{\partial u_j^{(l,t,p)}} \frac{\partial u_j^{(l,t,p)}}{\partial w_{ji}^{(l,t)}} \quad \text{より}$$

$$\delta_j^{(l,t)} = \sum_{p \in C^s} \sum_{k=1}^{N^{(l+1)}} \frac{\partial E_s^{(t,p)}}{\partial u_k^{(l+1,t,p)}} \frac{\partial u_k^{(l+1,t,p)}}{\partial x_j^{(l,t,p)}} \frac{\partial x_j^{(l,t,p)}}{\partial u_j^{(l,t,p)}}$$

$$= \sum_{k=1}^{N^{(l+1)}} \delta_k^{(l+1,t)} w_{kj}^{(l+1,t)} \frac{\partial x_j^{(l,t,p)}}{\partial u_j^{(l,t,p)}}$$

第 l 層の δ は次の層(第 $l + 1$ 層)の δ から求めることができる

→ 出力層(第 L 層)の δ を求めることができればそれよりも前の層の δ はそれを用いて求めることができる

$$E_s^{(t)} = \frac{1}{2} \sum_{p \in C^s} \sum_{k=1}^K \left(d_k^{(p)} - x_k^{o(p)} \right)^2$$

$$E_s^{(t,p)} = \frac{1}{2} \sum_{k=1}^K \left(d_k^{(p)} - x_k^{o(p)} \right)^2$$

$$\delta_k^{(L,t)} = \sum_{p \in C^s} \frac{\partial E_s^{(t,p)}}{\partial u_k^{(L)}}$$

$$= \sum_{p \in C^s} \sum_{k=1}^K (x_k^{(L,p)} - d_k^{(p)})$$

出力層の δ (交差エントロピー誤差(2値分類)の場合) 49

$$E_s^{(t)} = - \sum_{p \in C^s} [d^{(p)} \log x^{O(p)} + (1 - d^{(p)}) \log (1 - x^{O(p)})]$$

$$E_s^{(t,p)} = -[d^{(p)} \log x^{O(p)} + (1 - d^{(p)}) \log (1 - x^{O(p)})]$$

$$\begin{aligned} \delta^{(L,t)} &= \sum_{p \in C^s} \frac{\partial E_s^{(t,p)}}{\partial u^{(L)}} \\ &= \sum_{p \in C^s} (x^{(L,p)} - d^{(p)}) \end{aligned}$$

出力層の δ (交差エントロピー誤差(多クラス分類)の場合)

50

$$E_s^{(t)} = - \sum_{p \in C^s} \sum_{k=1}^K d_k^{(p)} \log x_k^{o(p)}$$

$$E_s^{(t,p)} = - \sum_{k=1}^K d_k^{(p)} \log x_k^{o(p)}$$

$$\begin{aligned} \delta_k^{(L,t)} &= \sum_{p \in C^s} \frac{\partial E_s^{(t,p)}}{\partial u_k^{(L)}} \\ &= \sum_{p \in C^s} \sum_{k=1}^K (x_k^{(L,p)} - d_k^{(p)}) \end{aligned}$$

δ の計算の途中経過は省略するが
いずれの誤差関数の場合も出力層の
出力と教師信号との差によって求められる

Step 1 : ネットワークの出力の計算 (順伝搬)

ミニバッチ内のデータ($p \in C^S$)を入力し、すべての層のニューロンの内部状態 $u^{(l,t,p)}$ と出力 $x^{(l,t,p)}$ を計算する。

Step 2 : 出力層での δ の計算

ミニバッチ内のすべてのデータに対して求めたネットワークの出力に対して誤差を求め、それに基づいて δ を計算する。

Step 3 : 出力層以外の δ の計算 (逆伝搬)

中間層での $\delta_j^{(l,t)}$ ($l = L - 1, \dots, 2$)を出力層に近い層から順に計算する。

Step 4 : 各層の重みの更新

Step 2, 3で求めた δ に基づいて重みに関する誤差関数の微分を

$$\frac{\partial E_s^{(t)}}{\partial w_{ji}^{(l,t)}} = \delta_j^{(l,t)} x_i^{(l-1,t)}$$

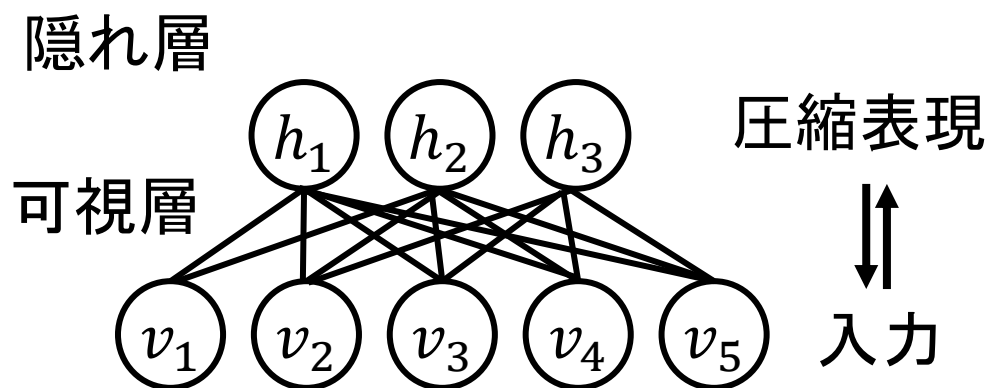
のように求め、これを用いて重みを

$$w_{ji}^{(l,t+1)} = w_{ji}^{(l,t)} - \eta \frac{\partial E_s^{(t)}}{\partial w_{ji}^{(l,t)}}$$

のように更新する。

可視層と隠れ層のニューロン間にのみ結合が存在するボルツマンマシン

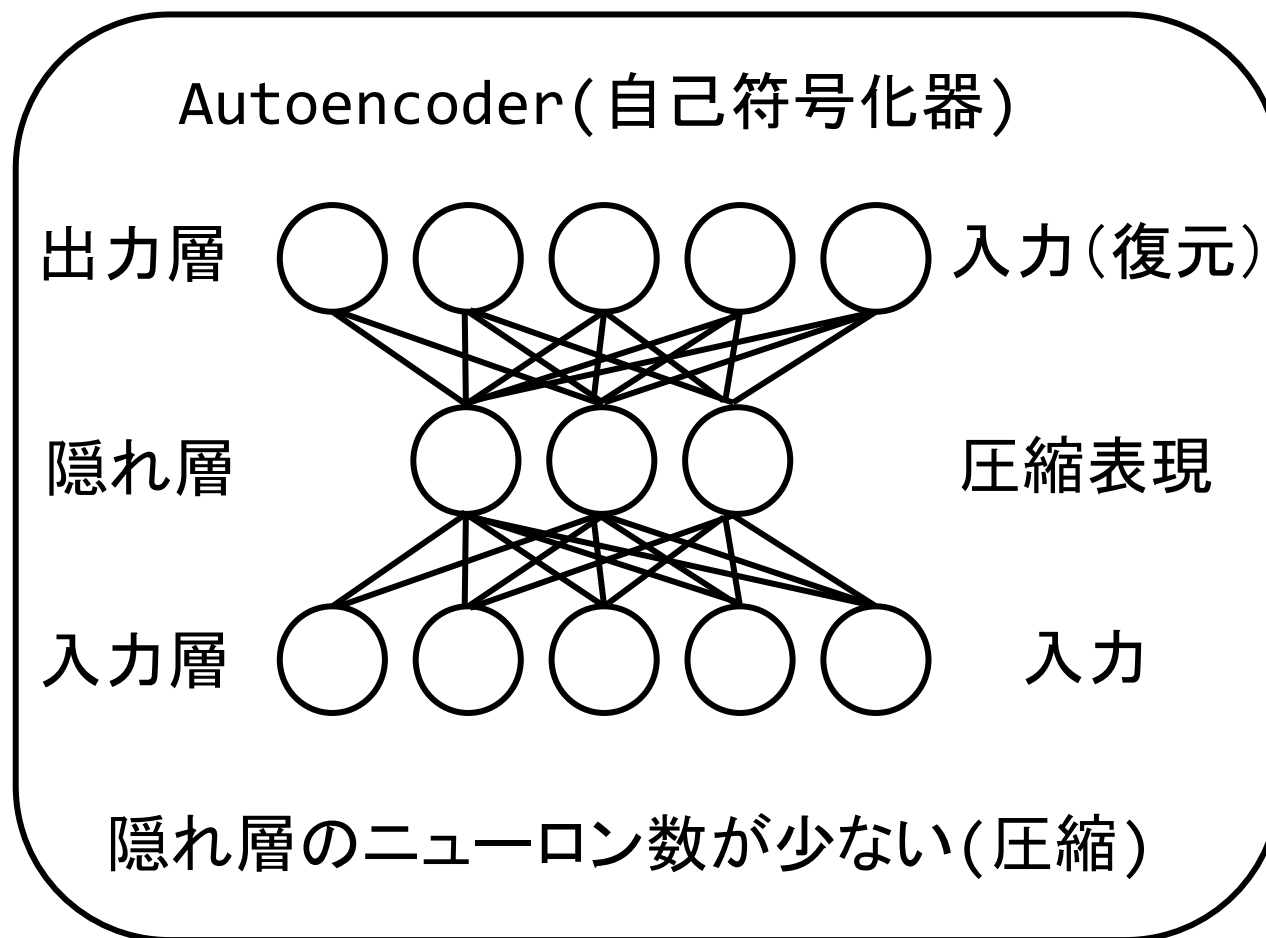
= 制約(restricted)



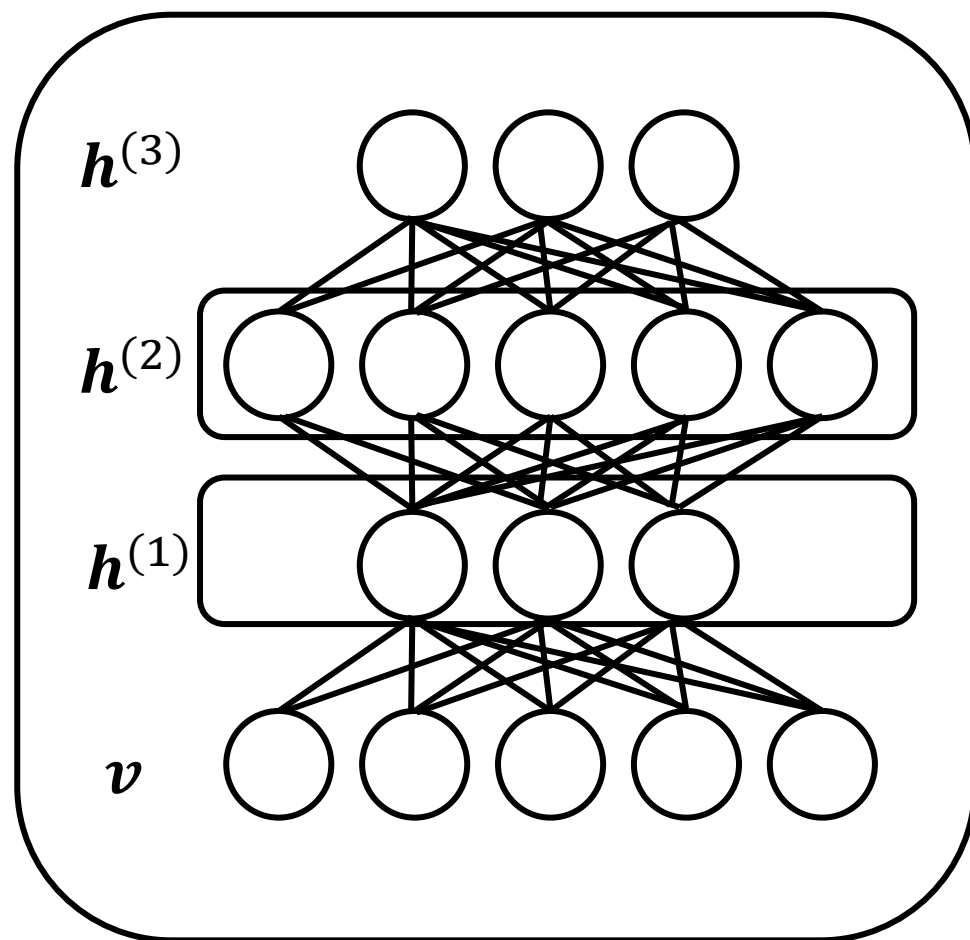
RBMを自己符号化器として使用

教師なし事前学習

Deep Learning が注目される
きっかけとなったアイデア

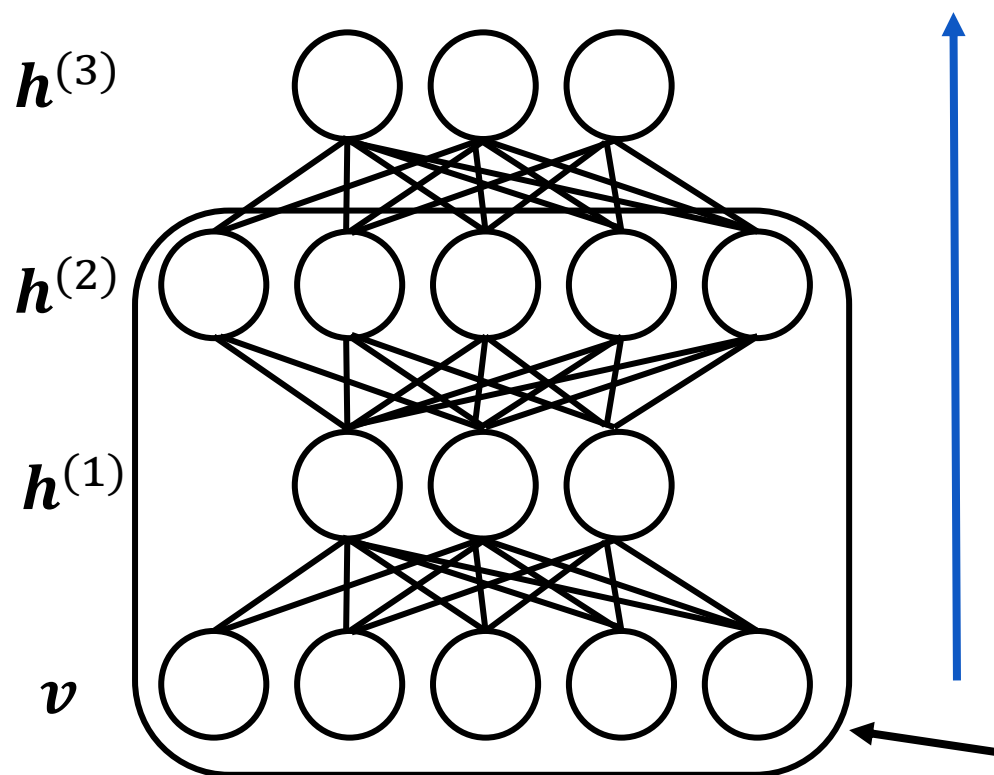


複数のRBMを階層構造に積み重ねたネットワーク



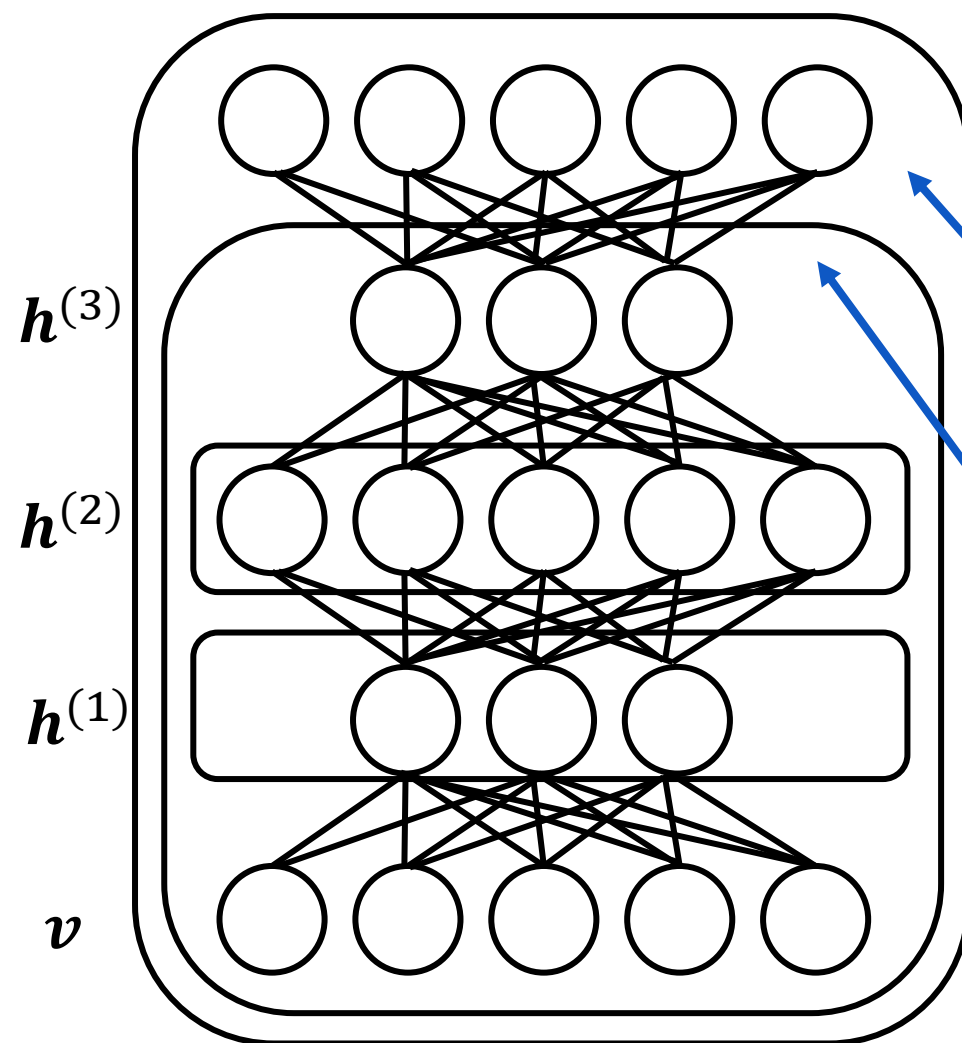
下の層から順番に学習

複数のRBMを階層構造に積み重ねたネットワーク



下の層から順番に学習

DBNではこの部分を有向グラフとして扱うが
DBMでは無向グラフとして扱う



各層をRBMとみなして事前学習(プレトレーニング)した値を重みの初期値として全体の重みを再度学習(微調整)

分類などに使う場合には
そのための出力層を追加

ここの重みは乱数で初期化